ENUNCIADO DE ACTIVIDAD / JAVASCRIPT VANILLA

1. Descripción y Objetivo General

Evaluar el dominio práctico de JavaScript Vanilla (ES6+), manipulación del DOM, consumo asíncrono de APIs REST externas y persistencia local de datos en el navegador mediante **localStorage** (Web Storage API). Los estudiantes desarrollarán una aplicación interactiva de gestión de tareas (**TaskManager**) funcional en modo *offline*.

2. Funcionalidades Obligatorias en JavaScript (4 Puntos)

El código debe ser escrito en JS sin librerías ni frameworks (React, Vue, jQuery):

Módulo	Funcionalidades Requeridas	Técnica JS Requerida
Gestión CRUD	Crear, editar, eliminar y marcar tareas como completadas. Validación estricta de campos vacíos desde el código JS.	Eventos DOM (addEventListener), arreglos u objetos.
Filtros y Búsqueda	Filtrar por estado (Pendientes, Completadas, Todas) y realizar búsqueda dinámica en tiempo real por título.	Métodos de Arrays (filter, includes, toLowerCase).
Ordenamiento	Ordenar la lista visible según la fecha de creación o el nivel de prioridad (Alta, Media, Baja).	Método sort() con funciones comparadoras.
Métricas y Dashboard	Panel interactivo que calcula y muestra en tiempo real: total de tareas, conteo de pendientes y completadas.	Actualización reactiva del DOM tras manipular los datos.

3. Consumo de API REST y Persistencia Local (4 Puntos)

Se utilizará la API pública de pruebas **JSONPlaceholder** (https://jsonplaceholder.typicode.com/todos):

Escenario	Comportamiento y Resultado Esperado
Carga Inicial API	Consultar tareas mediante fetch() y async/await. Si localStorage está vacío, importar los datos recibidos serializándolos con JSON.stringify() sin duplicar registros.
Manejo de Errores	Capturar fallos de red o de servidor mediante try/catch y mostrar una notificación visual amigable al usuario en pantalla.
Persistencia Local (localStorage)	Cualquier modificación (crear, editar, borrar) debe guardarse en el navegador con localStorage.setItem() y recuperarse con JSON.parse(localStorage.getItem()) al recargar o al trabajar sin internet.

4. Componente de Investigación Técnica (3 Puntos)

Para fomentar la autonomía y la capacidad de investigación en JavaScript, el estudiante deberá seleccionar e implementar **UNA** de las siguientes técnicas e incluir una breve explicación en el informe PDF:

Opciones de Investigación Técnicas (Elegir 1)

- **Opción A — Arquitectura Modular ES6:** Organizar la aplicación utilizando módulos JavaScript nativos (import / export), separando la interfaz visual (DOM), el servicio API y el módulo gestor de localStorage.
- **Opción B — Custom Events o Patrón Observador:** Implementar un sistema de eventos personalizados (CustomEvent) para notificar cambios de estado en las tareas y desencadenar la re-renderización automática del Dashboard de métricas.

5. Verificación de Autoría, GitHub y Declaración de IA (4 Puntos)

Para asegurar la autoría real del código y promover el uso ético y reflexivo de la Inteligencia Artificial, se establecen los siguientes requerimientos obligatorios:

- **Historial de Commits en GitHub (Obligatorio):** El repositorio debe reflejar el avance por etapas. Entregas hechas en un único commit (ej. "proyecto terminado") o subidas en un solo día tendrán una penalización de **-5 puntos**. Se requiere un mínimo de **5 commits significativos**.
- **Archivo de Declaración de IA (AI_DECLARATION.md):** Se debe incluir este archivo en la raíz del repositorio de GitHub respondiendo:
  a. ¿Qué herramientas de IA utilizó (ChatGPT, Claude, Copilot, etc.) y en qué momentos?
  b. Adjunte al menos **2 prompts** (instrucciones) clave que utilizó durante el desarrollo.
  c. ¿Qué partes del código sugerido por la IA contenían errores o requirieron ajustes manuales para integrarse correctamente a su proyecto?
- **Informe PDF Final:** Documento (máximo 3 páginas) con portada, enlace al repositorio público de GitHub, resumen explicativo del tema investigado y capturas de pantalla que demuestren las tareas guardadas en el panel *Application > Local Storage* de DevTools.